DR. RVR NRI INSTITUTE OF TECHNOLOGY

Pothavarappadu, Agiripalli, Eluru Dist.

PYTHON PROGRAMMING - ABL ASSIGNMENT REPORT

Name : Benny Bobbili

Roll No : 26000861

Year / Sem : I B.Tech - I Sem

Branch & Section : CSM - 1

Github Repo Name : Python-Activity-Based-Learning

Selected 10 Case Studies List :-

S.no	Case Study No	Topic Name	File name
1.	Case Study 8	Manufacturing Cost and Profitability Analysis	case1.py
2.	Case Study 12	Attendance Recovery Computation	case2.py
3.	Case Study 18	Fuel Consumption and Route Comparison	case3.py
4.	Case Study 33	Campus Registration Identifier Analyzer	case4.py
5.	Case Study 39	Multi-Stage Examination Eligibility System	case5.py
6.	Case Study 45	Weekly Electricity Consumption Matrix	case6.py
7.	Case Study 49	Tuple-Based Product Catalogue Audit	case7.py
8.	Case Study 51	Bus Passenger Occupancy Simulation	case8.py
9.	Case Study 53	Food Waste Monitoring System	case9.py
10.	Case Study 59	Campus Transport Allocation	case10.py

1) Case Study 8 : Manufacturing Cost and Profitability Analysis

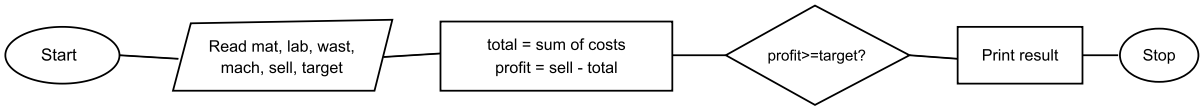
Problem Statement: Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.
Objectives: To find total cost and profit using basic operators and check if profit target is met using if-else.
Assumptions: All costs entered are positive numbers and total cost is sum of all expenses.

Algorithm :-
Step 1: Start
Step 2: Read mat, lab, wast, mach, sell, target
Step 3: Calculate total_cost = mat + lab + wast + mach
Step 4: Calculate profit = sell - total_cost
Step 5: If profit >= target print Target met, else print below target or loss
Step 6: Stop

Pseudocode:

```
BEGIN
INPUT mat, lab, wast, mach, sell, target
total_cost = mat + lab + wast + mach
profit = sell - total_cost
IF profit >= target THEN PRINT "Profit target MET"
ELSE PRINT "Not met"
END
```

Flowchart :



Python Code (case1.py) :

```
mat = float(input("Enter material cost: "))
lab = float(input("Enter labour charges: "))
wast = float(input("Enter wastage cost: "))
mach = float(input("Enter machine cost: "))
sell = float(input("Enter selling price: "))
target = float(input("Enter profit target: "))
total_cost = mat + lab + wast + mach
profit = sell - total_cost
print("Total Cost:", total_cost)
print("Profit:", profit)
if profit >= target:
    print("Profit target MET!")
elif profit > 0:
    print("Below target by:", target - profit)
else:
    print("Loss incurred.")
```

Output :

```
Enter material cost: 5000
Enter labour charges: 2000
Enter wastage cost: 500
Enter machine cost: 1500
Enter selling price: 12000
Enter profit target: 2500
Total Cost: 9000.0
Profit: 3000.0
Profit target MET!
```

Conclusion: The program calculates total manufacturing cost and checks if the profit target is reached.

2) Case Study 12 - Attendance Recovery Computation

Problem Statement - Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold.
Objective - To calculate attendance percentage and use while loop to find extra classes needed to get 75%.
Assumptions - Total classes is greater than 0 and attending next classes adds 1 to attended and total.

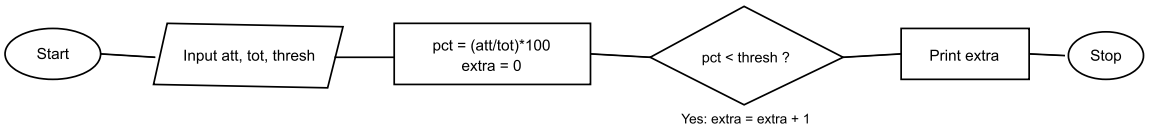
Algorithm:

1. Start
2. Input attended classes (att), total classes (tot), and required percentage (thresh)
3. Calculate curr_pct = (att / tot) * 100
4. Set extra = 0
5. Repeat while ((att + extra) / (tot + extra)) * 100 < thresh : extra = extra + 1
6. Print extra classes needed and Stop

Pseudocode:

```
BEGIN
INPUT att, tot, thresh
curr_pct = (att / tot) * 100
extra = 0
WHILE ((att + extra) / (tot + extra)) * 100 < thresh
   extra = extra + 1
END WHILE
PRINT extra
END
```

Flowchart :-



Source Code (case2.py):

```
att = int(input("Enter classes attended: "))
tot = int(input("Enter total classes held: "))
thresh = float(input("Enter required %: "))
curr_pct = (att / tot) * 100
print("Current Attendance:", round(curr_pct, 2), "%")
if curr_pct >= thresh:
    print("Already eligible! Extra needed: 0")
else:
    extra = 0
    while ((att + extra) / (tot + extra)) * 100 < thresh:
        extra = extra + 1
    print("Consecutive classes needed:", extra)
```

Test Case & Output:

```
Enter classes attended: 30
Enter total classes held: 50
Enter required %: 75
Current Attendance: 60.0 %
Consecutive classes needed: 30
```

Conclusion - We found the current attendance percentage and calculated how many classes to attend continuously using while loop.

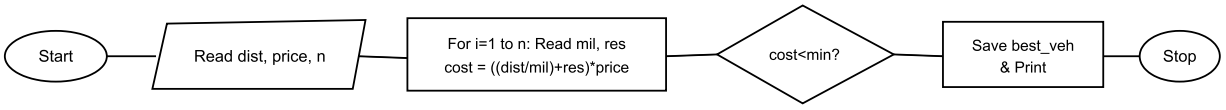
3. Case Study 18 : Fuel Consumption and Route Comparison

Problem Statement: Calculate fuel requirements and journey costs for several vehicles using distance, mileage, fuel prices, and reserve requirements; identify the lowest-cost feasible option.
Objectives: To calculate fuel needed and journey cost for each vehicle using for loop and find lowest cost vehicle.
Assumptions: Fuel needed is (distance / mileage) + reserve fuel.

Algorithm:
1. Start
2. Read distance, fuel price, and number of vehicles n
3. Take min_cost = 9999999.0 and best_veh = ""
4. Run for loop from 1 to n and read vehicle name, mileage, reserve
5. Calculate fuel = (dist / mil) + res and cost = fuel * price
6. If cost < min_cost then min_cost = cost and best_veh = v_name
7. Print best_veh and min_cost, Stop

Pseudocode:
```
BEGIN
INPUT dist, price, n
min_cost = 9999999
FOR i = 1 TO n DO
   INPUT v_name, mil, res
   fuel = (dist / mil) + res
   cost = fuel * price
   IF cost < min_cost THEN min_cost = cost, best_veh = v_name
END FOR
PRINT best_veh, min_cost
END
```

Flowchart:



Python Code (case3.py) :
```
dist = float(input("Enter distance (km): "))
price = float(input("Enter fuel price/L: "))
n = int(input("Enter number of vehicles: "))
min_cost = 9999999.0
best_veh = ""
for i in range(1, n + 1):
    v_name = input("Vehicle name: ")
    mil = float(input("Mileage (km/l): "))
    res = float(input("Reserve fuel (L): "))
    fuel = (dist / mil) + res
    cost = fuel * price
    print(v_name, "Cost:", round(cost, 2))
    if cost < min_cost:
        min_cost = cost
        best_veh = v_name
print("Best Vehicle:", best_veh, "Cost:", round(min_cost, 2))
```

Output :
```
Enter distance (km): 300
Enter fuel price/L: 100
Enter number of vehicles: 2
Vehicle name: Car, Mileage: 15, Reserve: 5 -> Car Cost: 2500.0
Vehicle name: Bike, Mileage: 50, Reserve: 2 -> Bike Cost: 800.0
Best Vehicle: Bike Cost: 800.0
```

Conclusion: The program calculates fuel and cost for vehicles and prints the cheapest vehicle for the route.

4) Case Study 33 : Campus Registration Identifier Analyzer

Problem Statement: Examine student identification strings, validate their structure, extract specified components, and classify records according to branch and admission year codes.
Objectives: To check if registration number is valid using string slicing and isdigit(), isalpha() methods.
Assumptions: Roll number has 7 characters like 26CSE45 (first 2 digits year, next 3 letters branch, last 2 digits roll no).

Algorithm :-
Step 1: Start and read number of IDs n
Step 2: Loop n times and input string reg
Step 3: Check if len(reg) == 7, otherwise print invalid length
Step 4: Slice yr = reg[0:2], br = reg[2:5], num = reg[5:7]
Step 5: Check yr.isdigit() and br.isalpha() and num.isdigit(), then print Year, Branch, Number
Step 6: Stop

Pseudocode :-

```
BEGIN
INPUT n
FOR i = 1 TO n DO
   INPUT reg
   IF len(reg) == 7 AND reg[0:2] is digit AND reg[2:5] is alpha AND reg[5:7] is digit THEN
      PRINT reg[0:2], reg[2:5], reg[5:7]
   ELSE PRINT "Invalid ID"
END FOR
END
```

Flowchart :



Python Program (case4.py):

```
n = int(input("Enter number of IDs: "))
for i in range(1, n + 1):
    reg = input("Enter ID (YYBBBNN): ").upper()
    if len(reg) != 7:
        print("INVALID: Length must be 7")
    else:
        yr = reg[0:2]
        br = reg[2:5]
        num = reg[5:7]
        if yr.isdigit() and br.isalpha() and num.isdigit():
            print("VALID -> Year: 20" + yr, "Branch:", br, "No:", num)
        else:
            print("INVALID: Wrong character format")
```

Output:

```
Enter number of IDs: 2
Enter ID (YYBBBNN): 26CSE45
VALID -> Year: 2026 Branch: CSE No: 45
Enter ID (YYBBBNN): 26CS01
INVALID: Length must be 7
```

Conclusion: Student registration strings were checked and separated into year, branch, and roll number using string slicing.

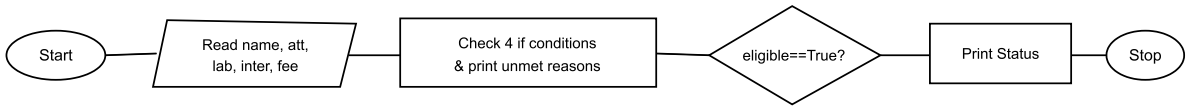
5) Case Study 39 - Multi-Stage Examination Eligibility System

Problem Statement: Determine eligibility for a final examination using attendance, laboratory completion, internal assessment, and fee-clearance conditions, reporting every unmet requirement.
Objectives: To check all 4 exam eligibility conditions using simple if statements and display all failed conditions.
Assumptions: Attendance should be >= 75, Lab = YES, Internal marks >= 20, and Fee = YES.

Algorithm:
1. Start
2. Read student name, att, lab, inter, fee
3. Take eligible = True
4. Check 4 separate if conditions for att < 75, lab != "YES", inter < 20, fee != "YES"
5. Print unmet condition and make eligible = False if any condition fails
6. Print final eligibility result and Stop

Pseudocode:
```
BEGIN
INPUT name, att, lab, inter, fee
eligible = True
IF att < 75 THEN PRINT "Low attendance", eligible = False
IF lab != "YES" THEN PRINT "Lab incomplete", eligible = False
IF inter < 20 THEN PRINT "Low internals", eligible = False
IF fee != "YES" THEN PRINT "Fee not paid", eligible = False
IF eligible == True THEN PRINT "Eligible" ELSE PRINT "Not Eligible"
END
```

Flowchart :



Python Code (case5.py) :
```
name = input("Enter Student Name: ")
att = float(input("Attendance %: "))
lab = input("Lab Completed (YES/NO): ").upper()
inter = float(input("Internal Marks: "))
fee = input("Fee Cleared (YES/NO): ").upper()
eligible = True
if att < 75.0:
    print("Unmet: Attendance < 75%")
    eligible = False
if lab != "YES":
    print("Unmet: Lab incomplete")
    eligible = False
if inter < 20.0:
    print("Unmet: Internals < 20")
    eligible = False
if fee != "YES":
    print("Unmet: Fee not cleared")
    eligible = False
if eligible:
    print(name, "is ELIGIBLE")
else:
    print(name, "is NOT ELIGIBLE")
```

Output:
```
Enter Student Name: Rahul
Attendance %: 70, Lab: YES, Internal Marks: 25, Fee Cleared: NO
Unmet: Attendance < 75%
Unmet: Fee not cleared
Rahul is NOT ELIGIBLE
```

Conclusion: The program checks all exam eligibility conditions and prints every condition that the student did not meet.

6) Case Study 45 : Weekly Electricity Consumption Matrix

Problem Statement: Process appliance-wise electricity readings across seven days, calculate daily and appliance totals, and identify consumption exceeding specified limits.
Objectives: To store 7 days electricity readings in a nested list and find row totals and column totals.
Assumptions: Readings are taken for 7 days for n appliances.

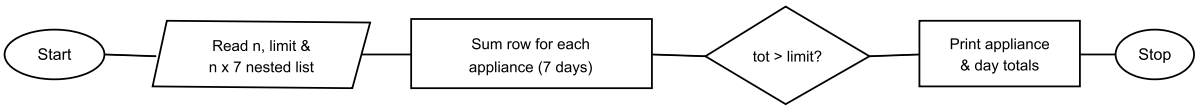
Algorithm :-

1. Start and read number of appliances n and weekly limit
2. Using nested for loops read 7 days units for each appliance into grid
3. Add each row to get appliance weekly total and check if it is greater than limit
4. Add each column from day 1 to 7 to get day-wise total consumption
5. Stop

Pseudocode:

```
BEGIN
INPUT n, limit
Read n x 7 nested list grid
FOR i = 0 TO n-1 DO
   sum row i and check if > limit
END FOR
FOR d = 0 TO 6 DO
   sum column d and print day total
END FOR
END
```

Flowchart :



Python Code (case6.py):

```
n = int(input("Enter number of appliances: "))
limit = float(input("Enter weekly limit: "))
grid = []
for i in range(n):
    row = []
    for d in range(7):
        u = float(input("App " + str(i+1) + " Day " + str(d+1) + ": "))
        row.append(u)
    grid.append(row)
for i in range(n):
    app_tot = 0.0
    for d in range(7):
        app_tot = app_tot + grid[i][d]
    if app_tot > limit:
        print("Appliance", i+1, "Total:", app_tot, "(EXCEEDED)")
    else:
        print("Appliance", i+1, "Total:", app_tot, "(OK)")
for d in range(7):
    day_tot = 0.0
    for i in range(n):
        day_tot = day_tot + grid[i][d]
    print("Day", d+1, "Total:", day_tot)
```

Output:

```
Enter number of appliances: 2, Enter weekly limit: 30
App 1 (7 days): 5, 5, 5, 5, 5, 5, 5 -> Appliance 1 Total: 35.0 (EXCEEDED)
App 2 (7 days): 2, 2, 2, 2, 2, 2, 2 -> Appliance 2 Total: 14.0 (OK)
Day 1 Total: 7.0 ... Day 7 Total: 7.0
```

Conclusion: Using nested lists, we calculated appliance weekly totals and daily electricity totals.

7) Case Study 49 - Tuple-Based Product Catalogue Audit

Problem Statement: Store product identifiers, prices, and stock quantities in tuples; detect invalid records, calculate inventory values, and identify items requiring replenishment.
Objectives: To store products as tuples in a list, unpack them, and check invalid records and low stock items.
Assumptions: Price <= 0 or stock < 0 is invalid. Stock less than 10 needs replenishment.

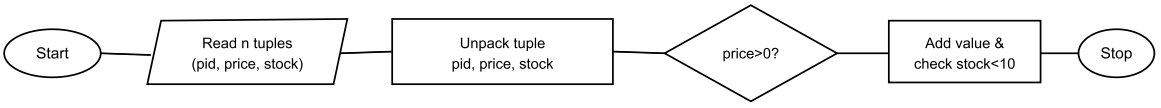
Algorithm:

1. Start and read number of products n
2. Read pid, price, stock and append tuple (pid, price, stock) into catalogue list
3. Loop through catalogue and unpack pid, price, stock
4. If price <= 0 or stock < 0 add pid to invalid_items
5. Else add price * stock to total_val, and if stock < 10 add pid to replenish_items
6. Print total value, invalid items, and replenish items. Stop

Pseudocode:

```
BEGIN
INPUT n
Read n tuples (pid, price, stock) into catalogue
total_val = 0
FOR each (pid, price, stock) in catalogue DO
   IF price <= 0 OR stock < 0 THEN append pid to invalid
   ELSE total_val += price * stock
        IF stock < 10 THEN append pid to replenish
END FOR
PRINT total_val, invalid, replenish
END
```

Flowchart :-



Python Code (case7.py):

```
n = int(input("Enter number of products: "))
catalogue = []
for i in range(n):
    pid = input("Product ID: ").upper()
    price = float(input("Unit Price: "))
    stock = int(input("Stock Qty: "))
    catalogue.append((pid, price, stock))
total_val = 0.0
invalid_items = []
replenish_items = []
for i in range(n):
    pid, price, stock = catalogue[i]
    if price <= 0 or stock < 0:
        invalid_items.append(pid)
    else:
        total_val = total_val + (price * stock)
        if stock < 10:
            replenish_items.append(pid)
print("Total Value:", total_val)
print("Invalid:", invalid_items)
print("Replenish:", replenish_items)
```

Output:

```
Enter number of products: 3
Product 1: P101, 500, 20 | Product 2: P102, 200, 5 | Product 3: P103, -100, 10
Total Value: 11000.0
Invalid: ['P103']
Replenish: ['P102']
```

Conclusion: Product details were stored in tuples and unpacked to find total inventory value and low stock items.

8) Case Study 51 : Bus Passenger Occupancy Simulation

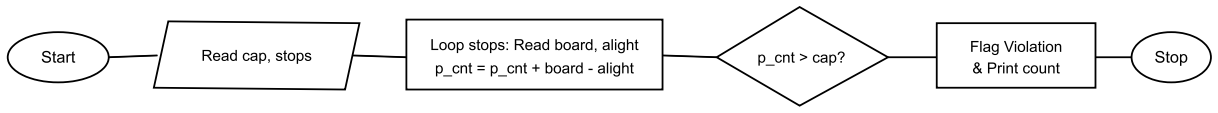
Problem Statement: Process boarding and alighting counts across multiple stops, calculate passenger counts at each stop, and flag any capacity violations.
Objectives: To track passenger count at each bus stop using a loop and check if passengers exceed bus capacity.
Assumptions: Initial passengers = 0. At every stop, current passengers = previous + boarding - alighting.

Algorithm:
Step 1: Start and read bus capacity (cap) and number of stops
Step 2: Set p_cnt = 0 and viol_cnt = 0
Step 3: Repeat for each stop s from 1 to stops: input boarding and alighting
Step 4: p_cnt = p_cnt + board - alight
Step 5: If p_cnt > cap, print overloaded alert and increment viol_cnt
Step 6: Print final passenger count and total violations. Stop

Pseudocode:

```
BEGIN
INPUT cap, stops
p_cnt = 0, viol_cnt = 0
FOR s = 1 TO stops DO
   INPUT board, alight
   p_cnt = p_cnt + board - alight
   IF p_cnt > cap THEN PRINT "Overloaded", viol_cnt = viol_cnt + 1
END FOR
PRINT p_cnt, viol_cnt
END
```

Flowchart :



Python Code (case8.py):

```
cap = int(input("Enter bus capacity: "))
stops = int(input("Enter number of stops: "))
p_cnt = 0
viol_cnt = 0
for s in range(1, stops + 1):
    board = int(input("Stop " + str(s) + " boarding: "))
    alight = int(input("Stop " + str(s) + " alighting: "))
    p_cnt = p_cnt + board - alight
    if p_cnt < 0:
        p_cnt = 0
    print("Stop", s, "Occupancy:", p_cnt)
    if p_cnt > cap:
        print("ALERT: Overloaded by", p_cnt - cap)
        viol_cnt = viol_cnt + 1
print("Final Passengers:", p_cnt)
print("Total Violations:", viol_cnt)
```

Output:

```
Enter bus capacity: 40, Enter number of stops: 3
Stop 1 boarding: 30, alighting: 0 -> Stop 1 Occupancy: 30
Stop 2 boarding: 20, alighting: 5 -> Stop 2 Occupancy: 45 (ALERT: Overloaded by 5)
Stop 3 boarding: 2, alighting: 15 -> Stop 3 Occupancy: 32
Final Passengers: 32 | Total Violations: 1
```

Conclusion: The program tracks passenger count at each stop and alerts whenever the bus is overloaded.

9) Case Study 53 - Food Waste Monitoring System

Problem Statement: Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.
Objectives: To calculate leftover food and wastage percentage daily and store high wastage days in a list.
Assumptions: Leftover = prepared - served, and wastage % = (leftover / prepared) * 100.

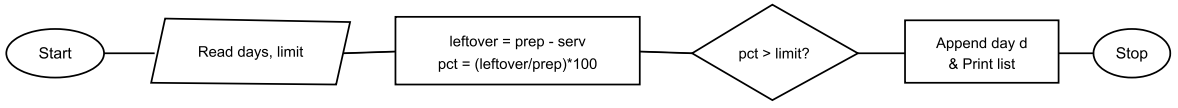
Algorithm :-

1. Start and read number of days and wastage limit %
2. Take empty list high_waste_days = []
3. Loop d from 1 to days and input prepared and served meals
4. Calculate leftover = prep - serv and waste_pct = (leftover / prep) * 100
5. If waste_pct > limit, append day d to high_waste_days
6. Print high_waste_days and Stop

Pseudocode:

```
BEGIN
INPUT days, limit
high_waste_days = []
FOR d = 1 TO days DO
   INPUT prep, serv
   leftover = prep - serv
   waste_pct = (leftover / prep) * 100
   IF waste_pct > limit THEN append d to high_waste_days
END FOR
PRINT high_waste_days
END
```

Flowchart :



Python Code (case9.py):

```
days = int(input("Enter number of days: "))
limit = float(input("Enter wastage % limit: "))
high_waste_days = []
for d in range(1, days + 1):
    prep = int(input("Day " + str(d) + " prepared: "))
    serv = int(input("Day " + str(d) + " served: "))
    leftover = prep - serv
    waste_pct = (leftover / prep) * 100
    print("Day", d, "Wastage:", round(waste_pct, 2), "%")
    if waste_pct > limit:
        high_waste_days.append(d)
print("Days exceeding limit:", high_waste_days)
```

Output:

```
Enter number of days: 3, Enter wastage % limit: 10
Day 1 prepared: 200, served: 190 -> Day 1 Wastage: 5.0 %
Day 2 prepared: 200, served: 160 -> Day 2 Wastage: 20.0 %
Day 3 prepared: 150, served: 140 -> Day 3 Wastage: 6.67 %
Days exceeding limit: [2]
```

Conclusion: Daily food wastage percentage was calculated and days crossing the limit were stored in a list.

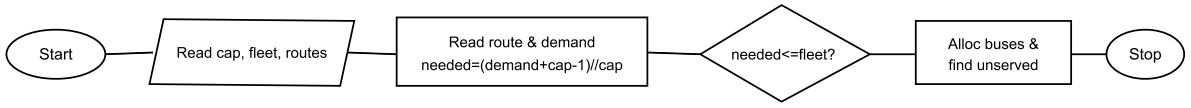
10) Case Study 59 : Campus Transport Allocation

Problem Statement: Determine the number of buses needed for several routes, account for bus capacities and passenger demand, and identify unserved passengers.
Objectives: To find number of buses required for each route using floor division (//) and find unserved passengers.
Assumptions: Buses needed = (demand + cap - 1) // cap.

Algorithm:
1. Start and read bus capacity (cap), available fleet, and number of routes
2. Loop for each route and input route name and passenger demand
3. Calculate needed = (demand + cap - 1) // cap
4. If needed <= fleet, allocate needed buses, unserved = 0, and reduce fleet
5. Else allocate remaining fleet buses, unserved = demand - (alloc * cap), and set fleet = 0
6. Print route allocation and total unserved passengers. Stop

Pseudocode:
```
BEGIN
INPUT cap, fleet, routes
FOR r = 1 TO routes DO
   INPUT r_name, demand
   needed = (demand + cap - 1) // cap
   IF needed <= fleet THEN alloc = needed, unserved = 0, fleet = fleet - alloc
   ELSE alloc = fleet, unserved = demand - (alloc * cap), fleet = 0
   PRINT r_name, needed, alloc, unserved
END FOR
END
```

Flowchart :



Python Code (case10.py):
```
cap = int(input("Enter bus capacity: "))
fleet = int(input("Enter available buses: "))
routes = int(input("Enter number of routes: "))
total_unserved = 0
for r in range(1, routes + 1):
    r_name = input("Route Name: ")
    demand = int(input("Passenger demand: "))
    needed = (demand + cap - 1) // cap
    if needed <= fleet:
        alloc = needed
        unserved = 0
        fleet = fleet - alloc
    else:
        alloc = fleet
        unserved = demand - (alloc * cap)
        fleet = 0
    total_unserved = total_unserved + unserved
    print(r_name, "-> Needed:", needed, "Alloc:", alloc, "Unserved:", unserved)
print("Remaining Buses:", fleet, "| Total Unserved:", total_unserved)
```

Output:
```
Enter bus capacity: 50, Enter available buses: 4, Enter number of routes: 2
Route Name: Vijayawada, Demand: 120 -> Vijayawada -> Needed: 3 Alloc: 3 Unserved: 0
Route Name: Guntur, Demand: 80 -> Guntur -> Needed: 2 Alloc: 1 Unserved: 30
Remaining Buses: 0 | Total Unserved: 30
```

Conclusion: The program calculates buses needed for each route and finds unserved passengers when buses are not enough.